Diseño basado en modelos de un controlador PI de temperatura para la planta térmica TCLab

Abner López Méndez, 2022273075 Jordi Segura Chinchilla, 2022240646
Escuela de Ingeniería Electrónica, Tecnológico de Costa Rica
EL-5409 — Laboratorio de Control Automático, II Semestre 2026
Profesor: Ing. Juan Luis Guerrero Fernández, PhD

***Resumen*—Se documenta el desarrollo de un sistema de control de temperatura para la planta térmica TCLab siguiendo un flujo de diseño basado en modelos. A partir de un balance de energía se obtuvo un modelo no lineal y su linealización alrededor de $Q_1 = 50\,\%$. Dos experimentos sobre la planta física, un escalón y una secuencia multinivel, permitieron identificar sendos modelos de primer orden con tiempo muerto; la validación cruzada seleccionó como nominal el modelo obtenido del ensayo escalón, $G_{p,\mathrm{nom}}(s) = 0{,}979\, e^{-8{,}00s}/(179{,}1s + 1)$, con un ajuste del 88,7 % sobre datos no empleados en su identificación. Las reglas SIMC con $\tau_c = 2\theta$ produjeron un controlador PI con $K_p = 7{,}62\,\%/^\circ\mathrm{C}$ y $T_i = 96\,\mathrm{s}$, que en simulación Model-in-the-Loop alcanza un tiempo de establecimiento de 205 s sin sobreimpulso al incorporar anti-windup por *clamping*, frente a 392 s y 30,1 % de sobreimpulso sin él. El código generado con Embedded Coder reprodujo exactamente la respuesta del modelo tanto en Software-in-the-Loop como en Processor-in-the-Loop sobre un Arduino Uno R4 Minima. La implementación final sobre la TCLab, ejecutada en el Arduino Leonardo mediante Monitor & Tune, alcanzó la referencia de 60 °C con un error estacionario de 0,04 °C y un tiempo de establecimiento de 298 s, con una ganancia estática aparente cerca de un 15 % menor que la del modelo nominal.**

***Index Terms*—TCLab, diseño basado en modelos, identificación FOPDT, SIMC, anti-windup, SIL, PIL, Embedded Coder.**

I. Introducción

La Temperature Control Lab (TCLab) es una planta térmica didáctica montada sobre un Arduino Leonardo. Dispone de dos transistores que actúan como calentadores y de dos sensores de temperatura adheridos a ellos; la potencia de cada calentador se comanda como un porcentaje de su valor máximo y la temperatura se mide en grados Celsius. En este trabajo se utilizó únicamente el primer par calentador–sensor, de modo que la variable manipulada es Q_1, expresada en porcentaje, y la variable controlada es T_1.

El problema de control consiste en llevar T_1 desde la temperatura ambiente hasta una referencia de 60 °C y mantenerla en ese valor, respetando el límite de potencia $Q_1 \in [0, 90]\,\%$ impuesto para proteger el equipo. Los requisitos de desempeño establecidos fueron un tiempo de establecimiento $t_s \leq 600\,\mathrm{s}$ y un sobreimpulso $M_p \leq 5\,\%$.

El objetivo del trabajo no se limita a obtener un controlador que funcione en simulación, sino a recorrer un flujo completo de diseño basado en modelos (*Model-Based Design*, MBD). En este enfoque el modelo ejecutable es el eje del desarrollo, y entre el diseño matemático y el sistema físico se intercalan etapas sucesivas de verificación que comprueban aspectos distintos de la solución [1]. El flujo seguido fue

$$\text{modelado} \rightarrow \text{identificación} \rightarrow \text{diseño} \rightarrow \text{MIL} \rightarrow \text{código} \rightarrow \text{SIL} \rightarrow \text{PIL} \rightarrow \text{implementación física.}$$

Cada etapa acota el riesgo de la siguiente. La evaluación *Model-in-the-Loop* (MIL) comprueba el desempeño del algoritmo contra un modelo de la planta; *Software-in-the-Loop* (SIL) verifica que el código generado se comporte como el modelo del que proviene; *Processor-in-the-Loop* (PIL) verifica que ese código conserve su comportamiento al ejecutarse sobre el procesador objetivo; y solo entonces el controlador se cierra sobre la planta real. El documento sigue ese mismo orden.

II. Modelado e identificación del sistema

II-A. Modelo físico

El conjunto formado por el transistor calentador y el sensor se representó como una única masa a temperatura uniforme. Bajo esa hipótesis, y considerando pérdidas por convección natural y por radiación hacia el entorno, el balance de energía conduce a [2]

$$mc_p \frac{dT}{dt} = \alpha Q - UA\,(T - T_a) - \epsilon\sigma A\,(T^4 - T_\infty^4), \qquad (1)$$

donde α convierte el porcentaje de potencia comandado en vatios entregados al sistema, UA es el coeficiente global de convección, y ϵ y σ son la emisividad y la constante de Stefan–Boltzmann. Las hipótesis adoptadas fueron masa concentrada y temperatura uniforme, convección predominantemente natural, potencia del calentador proporcional al comando, y $T_a \approx T_\infty$ por suponer el equipo y el recinto inicialmente en equilibrio térmico. Las temperaturas del término radiativo se manejaron en kelvin.

La no linealidad de (1) proviene exclusivamente del término T^4. Linealizando alrededor de un punto de operación $(\bar{T}, \bar{Q})$ y definiendo las variables de desviación $\Delta T = T - \bar{T}$ y $\Delta Q = Q - \bar{Q}$, se obtiene [3]

$$\frac{d\,\Delta T}{dt} = \gamma\,\Delta T + \beta\,\Delta Q, \qquad (2)$$

$$\gamma = -\frac{UA + 4\epsilon\sigma A\bar{T}^3}{mc_p}, \qquad \beta = \frac{\alpha}{mc_p}, \qquad (3)$$

cuya función de transferencia es de primer orden,

TABLA I
MODELO FÍSICO LINEALIZADO EN $\bar{Q} = 50\,\%$ Y ERROR DE LA APROXIMACIÓN LINEAL FRENTE AL MODELO NO LINEAL.

Magnitud	**Valor**
Temperatura de equilibrio $\bar{T}$	60,47 °C
Pérdida por convección en equilibrio	0,2188 W (43,8 %)
Pérdida por radiación en equilibrio	0,2811 W (56,2 %)
γ	$-7{,}548 \times 10^{-3}$/s
β	$5{,}000 \times 10^{-3}$ °C/(% s)
Constante de tiempo τ	132,5 s
Ganancia estática K	0,6625 °C/%
Error del modelo lineal frente al no lineal	
$\Delta Q = 5\,\%$: error final / RMSE	0,029 °C / 0,022 °C
$\Delta Q = 40\,\%$: error final / RMSE	1,744 °C / 1,337 °C

$$G(s) = \frac{K}{\tau s + 1}, \qquad \tau = -\frac{1}{\gamma}, \qquad K = -\frac{\beta}{\gamma}. \tag{4}$$

En (2) el coeficiente γ, dado por (3), reúne las dos vías de pérdida, y es el término radiativo el que lo hace depender del punto de operación. El punto empleado fue $\bar{Q} = 50\,\%$; los valores de τ y K que resultan de (4) se resumen en la Tabla I.

Los dos últimos renglones de la Tabla I delimitan el alcance de la aproximación lineal. Ante una perturbación de 5 % el error no llega a 0,03 °C, mientras que al multiplicar la amplitud por ocho el error crece casi sesenta veces. Este crecimiento no es proporcional porque el término omitido es de cuarto orden en la temperatura, y en el punto de operación la radiación ya aporta el 56,2 % de las pérdidas totales. Es decir, el modelo lineal es útil para excursiones pequeñas alrededor del punto de operación, pero no para describir el calentamiento completo desde temperatura ambiente, que es justamente el transitorio que debe manejar el controlador. Esta limitación motivó obtener el modelo de diseño de forma experimental en lugar de usar los parámetros físicos.

II-B. Identificación experimental

Se realizaron dos experimentos sobre la unidad TCLab n.º 4, con temperatura ambiente de 24 °C y periodo de muestreo $T_s = 1\,\text{s}$. El primero aplicó un escalón de Q_1 de 0 % a 50 % y se registró durante 1860 s (1861 muestras), partiendo de $T_1 = 24{,}78\,^\circ\text{C}$. El segundo aplicó una secuencia multinivel con escalones sucesivos de 10 % a 50 % durante 3520 s (3521 muestras), partiendo de $T_1 = 25{,}27\,^\circ\text{C}$. El ensayo escalón concentra la información en una sola transición amplia, mientras que el multinivel recorre varios niveles de potencia y permite observar si la ganancia del proceso depende de la región de operación.

Ambos conjuntos se ajustaron a un modelo de primer orden con tiempo muerto (FOPDT) [4],

$$G_p(s) = \frac{K\, e^{-\theta s}}{\tau s + 1}, \tag{5}$$

expresado en variables de desviación, con $[K] = {}^\circ\text{C}/\%$ y $[\tau] = [\theta] = \text{s}$. Los parámetros de (5) se obtuvieron por dos vías: el procedimiento de mínimos cuadrados sobre la respuesta simulada empleado en la práctica, y la función `procest` de *System Identification Toolbox* con estructura `P1D`. La Tabla II recoge los cuatro modelos y la Fig. 1 muestra las señales experimentales junto con la respuesta de los modelos identificados.

TABLA II
PARÁMETROS FOPDT IDENTIFICADOS POR ENSAYO Y POR MÉTODO.

Ensayo	**Método**	K (°C/%)	τ (s)	θ (s)	Ajuste (%)
Escalón	Práctica	0,979	179,1	8,00	93,0
Escalón	`procest` P1D	0,979	178,7	8,36	93,0
Multinivel	Práctica	1,036	189,5	2,00	94,5
Multinivel	`procest` P1D	1,036	189,2	2,80	94,5

TABLA III
VALIDACIÓN CRUZADA. MÉTRICAS CALCULADAS DESDE EL PRIMER CAMBIO DE Q_1.

Modelo → datos	RMSE (°C)	MAE (°C)	Sesgo (°C)	R^2	Ajuste (%)
Escalón → multinivel	1,62	1,51	1,50	0,987	88,7
Multinivel → escalón	2,63	2,41	−2,41	0,934	74,4

Los dos métodos coinciden en K y τ hasta la tercera cifra significativa, lo que respalda la identificación. La única diferencia apreciable está en θ: el procedimiento de la práctica cuantiza el tiempo muerto a un número entero de periodos de muestreo, mientras que `procest` lo estima como variable continua. Las diferencias resultantes, 0,36 s y 0,80 s, son menores que un periodo de muestreo y por tanto no distinguibles con los datos disponibles.

La coincidencia entre ambos métodos depende de entregar los datos en variables de desviación: con las señales absolutas la estimación converge a $K \approx 1{,}47\,^\circ\text{C}/\%$ y $\theta = 0$, valor que no es la ganancia del proceso sino el cociente $T_1(\infty)/\Delta Q_1$, porque el estimador debe explicar también el nivel inicial de temperatura.

II-C. Validación cruzada y selección del modelo

Cada modelo se simuló con la entrada del experimento que no se usó para identificarlo, manteniendo fijos K, τ y θ. La Tabla III reporta las métricas calculadas desde el primer cambio de Q_1, que es el intervalo que evalúa la respuesta dinámica, y la Fig. 2 muestra el cruce del modelo seleccionado junto con su residuo.

El modelo del ensayo escalón predice claramente mejor los datos independientes: su RMSE es un 38 % menor y su ajuste es 14,3 puntos porcentuales mayor. Ninguno de los dos cruces está libre de sesgo. El modelo escalón subestima la temperatura en promedio 1,50 °C, y el residuo de la Fig. 2 crece de forma ordenada con el nivel de potencia: 1,29, 0,88, 1,23, 1,85 y 2,20 °C para Q_1 de 10 a 50 %. El modelo multinivel presenta el sesgo opuesto, −2,41 °C, es decir, sobreestima la temperatura del ensayo escalón. Ambos patrones apuntan al mismo origen: la ganancia efectiva de la planta no es constante, de modo que un modelo identificado en una región tiende a errar sistemáticamente en la otra. Esto es coherente

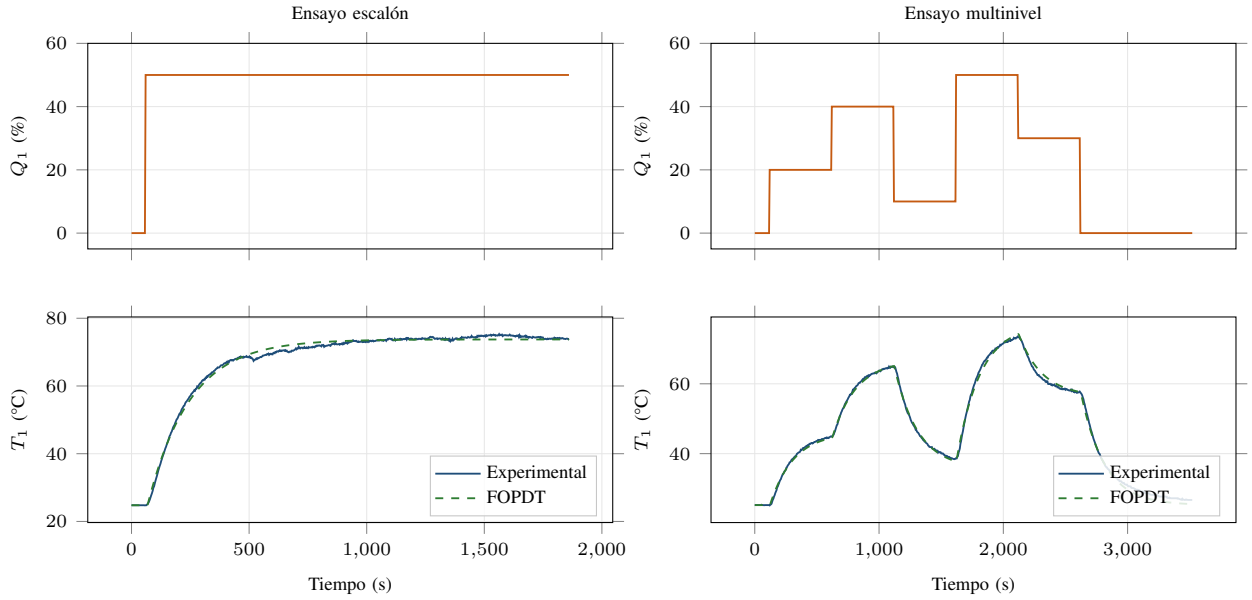


Figura 1. Señales de los dos experimentos de identificación y respuesta de los modelos FOPDT ajustados a cada uno. Izquierda: escalón de Q_1 a 50 %. Derecha: secuencia multinivel de 10 % a 50 %.

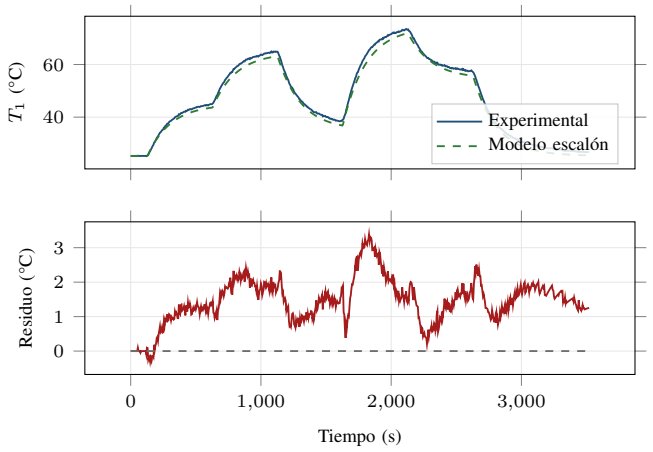


Figura 2. Validación del modelo identificado con el ensayo escalón sobre los datos multinivel, y residuo de predicción $e(k) = T_{1,\mathrm{exp}}(k) - T_{1,\mathrm{mod}}(k)$.

con lo observado en la Sección II-A, donde el peso creciente del término radiativo hacía que una única recta no describiera todo el rango.

Dado que ninguno de los dos modelos es válido en todo el rango, el criterio de selección fue la región de operación prevista para el controlador. La referencia de trabajo es 60 °C, que corresponde aproximadamente a $Q_1 = 50\,\%$, justamente la condición del ensayo escalón. Ese modelo además tuvo mejor capacidad predictiva sobre datos independientes. Su tiempo muerto, $\theta = 8{,}00\,\mathrm{s}$, equivale a ocho periodos de muestreo, lo que permite representar el retardo de forma exacta en la implementación discreta. El modelo nominal adoptado es entonces

$$G_{p,\mathrm{nom}}(s) = \frac{0{,}979\; e^{-8{,}00\,s}}{179{,}1\,s + 1} \quad [^\circ\mathrm{C}/\%]\,. \tag{6}$$

III. Diseño del controlador y evaluación MIL

III-A. Diseño del controlador PI

Se empleó un controlador PI en forma paralela continua,

$$G_c(s) = K_p + \frac{K_i}{s} = K_p\left(1 + \frac{1}{T_i s}\right), \qquad K_i = \frac{K_p}{T_i}, \tag{7}$$

empleada en ambos diseños, con $e = T_{\mathrm{ref}} - T_1$ en °C y salida en porcentaje de potencia, de modo que $[K_p] = \%/^\circ\mathrm{C}$, $[T_i] = \mathrm{s}$ y $[K_i] = \%/(^\circ\mathrm{C\,s})$. La acción integral es necesaria porque la planta no tiene integrador propio y se requiere error estacionario nulo ante una referencia constante.

El primer diseño utilizó las reglas SIMC [5], que para un modelo FOPDT dan

$$K_p = \frac{1}{K}\,\frac{\tau}{\tau_c + \theta}, \qquad T_i = \mathrm{mín}\{\tau,\; 4(\tau_c + \theta)\}, \tag{8}$$

donde τ_c es la constante de tiempo deseada en lazo cerrado. Se fijó $\tau_c = 2\theta$, un valor conservador que prioriza robustez frente a rapidez. Con los parámetros de (6) resulta $\tau_c = 16\,\mathrm{s}$ y $\tau_c + \theta = 24\,\mathrm{s}$; como $4(\tau_c + \theta) = 96\,\mathrm{s} < \tau$, el tiempo integral queda truncado por la segunda rama de (8).

El segundo diseño se obtuvo con *Control System Designer* sobre la misma planta nominal, usando una estrategia distinta: en lugar de aplicar una fórmula de sintonización se moldeó la respuesta en frecuencia del lazo abierto añadiendo un integrador y un cero real, y ajustando la ganancia hasta cumplir dos criterios propios más exigentes que los de SIMC, margen de fase mayor a 70° y margen de ganancia mayor a 20 dB. La

TABLA IV
CONTROLADORES DISEÑADOS Y SU COMPORTAMIENTO SOBRE EL MODELO LINEAL NOMINAL.

Parámetro	**PI–SIMC**	**PI–CSD**
K_p (%/°C)	7,62	3,00
K_i (%/(°C s))	0,0794	0,0200
T_i (s)	96	150
Frecuencia de cruce (rad/s)	0,0425	0,0168
Margen de fase (°)	64,2	79,1
Margen de ganancia (dB)	13,3	21,5
t_s lineal (s)	202,3	161,9
M_p lineal (%)	7,76	1,67

TABLA V
MÉTRICAS MIL. LAS TRES PRIMERAS COLUMNAS CORRESPONDEN AL PI–SIMC; LA ÚLTIMA, AL PI–CSD BAJO IDÉNTICAS CONDICIONES DE SIMULACIÓN.

Métrica	**Ideal**	**Sin AW**	**Con AW**	**CSD**
Tiempo de subida (s)	25,2	74,6	84,5	133,7
t_s (s)	202,3	391,7	205,0	251,9
M_p (%)	7,76	30,1	0,00	0,00
e_{ss} (°C)	0,00	0,00	0,00	0,00
$T_{1,\max}$ (°C)	62,8	70,8	60,0	60,0
Q_1 mínimo aplicado (%)	34,7	28,7	36,8	36,8
Q_1 máximo aplicado (%)	297,4	90,0	90,0	90,0
Tiempo con Q_1 saturada (s)	—	127	66	21

herramienta aproxima el retardo mediante Padé para trazar el lugar de las raíces; los márgenes y las métricas reportadas se recalcularon con el retardo exacto para evitar que esa aproximación contaminara la comparación.

Los dos controladores resultantes, en la forma (7), son

$$G_{c,\mathrm{SIMC}}(s) = 7{,}62 + \frac{0{,}0794}{s}, \quad G_{c,\mathrm{CSD}}(s) = 3{,}00 + \frac{0{,}0200}{s}, \tag{9}$$

con K_p en %/°C y K_i en %/(°C s). La Tabla IV resume ambos diseños y su comportamiento sobre el modelo nominal.

Sobre el modelo lineal, sin restricciones de actuador, el controlador obtenido con *Control System Designer* se ve mejor en ambas métricas y además tiene márgenes mayores. Ese resultado, sin embargo, no considera el límite de potencia, que es precisamente lo que evalúa la etapa siguiente.

III-B. Evaluación Model-in-the-Loop

El controlador PI–SIMC de (9) se implementó de forma discreta con $T_s = 1\,\mathrm{s}$ e integración por Euler hacia adelante,

$$I[k+1] = I[k] + K_i T_s\, e[k], \qquad Q_{\mathrm{cmd}}[k] = K_p\, e[k] + I[k], \tag{10}$$

distinguiendo la ganancia integral continua K_i del incremento por muestra $K_i T_s$. La planta se representó con (6) discretizada, con el retardo como ocho muestras de atraso. Se simularon tres configuraciones bajo idénticas condiciones —misma referencia, misma planta, misma condición inicial, $T_{\mathrm{amb}} = 24\,^{\circ}\mathrm{C}$, $T_{\mathrm{ref}} = 60\,^{\circ}\mathrm{C}$ y 1800 s de simulación—: PI ideal sin restricciones, PI con saturación $Q_1 \in [0, 90]\,\%$ sin anti-windup, y PI con saturación y anti-windup por *clamping*, que congela el integrador mientras la salida está saturada y el error empuja en la dirección de la saturación [6]. Los resultados se muestran en la Fig. 3 y la Tabla V; las métricas se calcularon con `stepinfo` [7].

El PI ideal cumple los requisitos en simulación, pero exige un máximo de 297 % de potencia. Ese resultado es físicamente irrealizable y muestra que evaluar el controlador sin las restricciones del actuador no permite concluir nada sobre su comportamiento real.

Al introducir la saturación sin anti-windup el desempeño se degrada de forma marcada: el sobreimpulso pasa de 7,76 % a 30,1 % y el tiempo de establecimiento casi se duplica. El mecanismo es visible en la Fig. 3. Durante los primeros 127 s el comando permanece por encima del límite, de modo que el error sigue integrándose aunque la potencia aplicada ya no pueda aumentar. Cuando T_1 cruza la referencia en $t = 103\,\mathrm{s}$ el integrador acumula un excedente que el error negativo debe descargar antes de que el comando vuelva al rango útil, lo que ocurre 24 s después. La planta entretanto sigue recibiendo el 90 % de potencia y alcanza 70,8 °C. Para recuperarse, el controlador debe reducir la potencia hasta el 28,7 % en $t = 186\,\mathrm{s}$, por debajo del 36,8 % que la planta necesita en régimen.

Con *clamping* el integrador deja de acumular mientras la salida está saturada, la saturación dura 66 s en lugar de 127 s y la respuesta llega a la referencia sin sobrepasarla. De las tres configuraciones, solo esta cumple simultáneamente $t_s \le 600\,\mathrm{s}$ y $M_p \le 5\,\%$; la configuración sin anti-windup falla el requisito de sobreimpulso por un factor de seis. Conviene notar que el anti-windup cuesta 10 s de tiempo de subida frente al caso sin protección, un precio bajo comparado con lo que evita.

La última columna de la Tabla V incorpora el PI–CSD con saturación y anti-windup, bajo la misma planta, el mismo T_s, los mismos límites, la misma condición inicial y el mismo tiempo de simulación.[1] Ambos controladores cumplen los requisitos y ambos alcanzan la referencia sin sobreimpulso y sin error estacionario, de modo que la comparación se decide por rapidez y por esfuerzo de control. El PI–SIMC establece 47 s antes, pero pide un comando inicial de 274 % y mantiene el actuador saturado 66 s; el PI–CSD pide 108 % y satura solo 21 s, a cambio de un tiempo de subida 57 % mayor. Es decir, el PI–CSD opera mucho más cerca de la capacidad real del actuador y tiene márgenes de estabilidad mayores, mientras que el PI–SIMC aprovecha mejor la autoridad disponible.

Ninguno de los dos domina al otro. Como ambos satisfacen las especificaciones con holgura y el tiempo de establecimiento no es crítico en esta planta, un criterio de robustez favorecería al PI–CSD. Para las etapas de generación y verificación de código se utilizó el PI–SIMC con saturación y anti-windup, según lo establecido en la guía de laboratorio, y el PI–CSD se conservó como contraste metodológico.

[1] En esa corrida la columna PI–SIMC da $t_r = 84{,}56\,\mathrm{s}$ y $t_s = 205{,}18\,\mathrm{s}$; la diferencia respecto de la tercera columna es de interpolación sub-muestra de `stepinfo` y no de comportamiento.

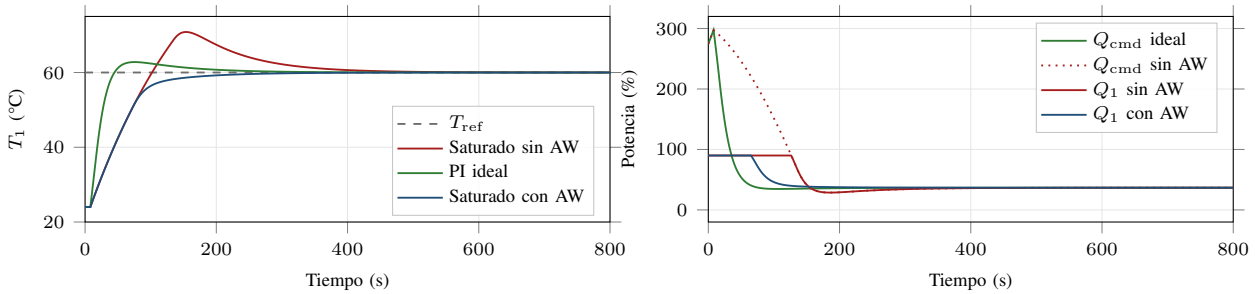


Figura 3. Evaluación MIL del PI–SIMC en tres configuraciones. Izquierda: temperatura. Derecha: potencia comandada Q_{cmd} y potencia aplicada Q_1. Se muestran los primeros 800 s de los 1800 s simulados.

Figura de `Controller.slx`

Figura 4. Modelo `Controller.slx`: implementación discreta del PI–SIMC con saturación y anti-windup por *clamping*.

IV. Generación de código y verificación de software

IV-A. Implementación del controlador

El sistema bajo prueba (*System Under Test*, SUT) es el modelo `Controller.slx`, que contiene únicamente el algoritmo de control. Su interfaz es $T_{\text{ref}}, T_1 \rightarrow Q_1$: recibe la referencia y la temperatura medida, ambas en °C, y entrega la potencia comandada en porcentaje. Internamente ejecuta la ley discreta (10) con $T_s = 1\,\text{s}$, aplica la saturación $Q_1 \in [0, 90]\,\%$ y el anti-windup por *clamping*. La Fig. 4 muestra su estructura.

PENDIENTE: Insertar aquí la captura de `Controller.slx` (Simulink) como `figuras/controller.png`. Es la única captura que el instructivo pide explícitamente.

La planta no forma parte de este modelo. Esa separación es deliberada: es lo que permite generar código solo del algoritmo, instanciarlo varias veces dentro de un banco de pruebas y sustituir una instancia por su versión compilada sin alterar el resto del lazo. Para ello el modelo se configuró con `Code interface packaging = Reusable function` y se habilitaron múltiples instancias por modelo de nivel superior.

De la configuración de generación de código solo interesan aquí tres decisiones. La primera es el uso de `ert.tlc` con lenguaje C bajo el estándar C99, que produce código orientado a sistemas embebidos [8]. La segunda es el solver de paso fijo discreto con paso T_s, coherente con (10). La tercera es `Default parameter behavior = Inlined` combinado con la clase de almacenamiento `ExportedGlobal` para Kp_{SIMC} y Ki_{SIMC}: el resto de las constantes se incrusta en el código, pero las dos ganancias permanecen como variables globales y por tanto siguen siendo calibrables sin regenerar. Para la etapa SIL el hardware objetivo fue la propia computadora (Intel x86-64), no todavía el Arduino.

TABLA VI
Correspondencia entre el modelo y el código generado.

Característica	Evidencia en el código
Interfaz	`void Controller(const real_T *rtu_T_ref, const real_T *rtu_T1, real_T *rty_Q1, DW_Controller_f_T *localDW)`
Estado del integrador	Campo único `Integrator_DSTATE` dentro de la estructura de estado `DW_Controller_f_T`
Parámetros calibrables	`real_T Kp_SIMC = 7.620337;` y `real_T Ki_SIMC = 0.079379;` definidos como variables globales
Saturación	Ramas explícitas con los límites `90.0` y `0.0` sobre la salida
Anti-windup	Condición que anula el incremento integral cuando hay exceso y su signo coincide con el del error
Tipo de dato	`typedef double real_T`
Separación controlador–planta	El archivo generado no contiene ningún modelo de la planta

Las comprobaciones previas a la generación —*Code Generation Advisor* con los objetivos *Execution efficiency* y *Traceability*, y *Model Advisor*— no reportaron ningún fallo. La única advertencia relevante se refiere al soporte de valores no finitos en el entorno de software, que no afecta al algoritmo implementado; no se aplicaron correcciones automáticas para eliminarla.

IV-B. Inspección del código generado

La revisión del código C verificó que las características esenciales del controlador sobrevivieran a la generación. La Tabla VI resume la correspondencia encontrada.

El punto menos evidente es el anti-windup, porque en el código no aparece como un bloque sino como una condición aritmética. El fragmento correspondiente es

```
if ((rtb_DeadZone != 0.0) && (tmp == tmp_0)) {
  rtb_IntegralGain = 0.0;
}
localDW->Integrator_DSTATE += rtb_IntegralGain;
```

La variable `rtb_DeadZone` mide cuánto excede el comando el rango permitido y vale cero mientras no hay satu-

ración; `tmp` y `tmp_0` codifican los signos del exceso y del término integral. Cuando hay saturación y ambos signos coinciden, el incremento se fuerza a cero y el estado del integrador no cambia. Esa es exactamente la lógica de *clamping* evaluada en MIL.

IV-C. Verificación Software-in-the-Loop

En SIL el código generado se compila y ejecuta sobre la computadora dentro de la misma simulación [9]. El banco `TCLab_Test_SIL.slx` contiene dos ramas que comparten referencia, temperatura ambiente, modelo de planta, condición inicial, periodo de muestreo y tiempo de simulación; la única diferencia es que una instancia del controlador se ejecuta en modo Normal y la otra en modo SIL. La comparación es por tanto *back-to-back*: misma entrada, dos implementaciones del mismo algoritmo.

Las diferencias evaluadas fueron

$$\Delta Q_1 = Q_{1,\mathrm{SIL}} - Q_{1,\mathrm{Normal}}, \qquad \Delta T_1 = T_{1,\mathrm{SIL}} - T_{1,\mathrm{Normal}}, \tag{11}$$

sobre las 1801 muestras de los 1800 s simulados. Ambas resultaron idénticamente nulas, como se observa en la Fig. 5 y se cuantifica en la Tabla VII, de modo que la prueba satisface el criterio de equivalencia con igualdad exacta y no solo dentro de una tolerancia.

Este resultado verifica que la traducción de modelo a código C no introdujo diferencias numéricas: el orden de las operaciones, la actualización del estado y la lógica de saturación y anti-windup se conservaron. No verifica, en cambio, nada relacionado con el procesador final, porque el código se compiló para la arquitectura del computador anfitrión y con aritmética `double`. Tampoco dice nada sobre la planta, que sigue siendo un modelo.

V. Verificación Processor-in-the-Loop

En PIL el código del controlador se compila para el procesador objetivo, se descarga a la tarjeta y se ejecuta allí, mientras Simulink intercambia con ella las señales de entrada y salida en cada periodo de muestreo [10]. El procesador empleado fue un Arduino Uno R4 Minima, utilizado exclusivamente como plataforma de verificación y no como el destino final del controlador.

El reparto entre lo que se ejecuta físicamente y lo que permanece simulado es el punto clave de esta etapa. Sobre el Arduino se ejecuta únicamente el algoritmo de control: la ley PI discreta, la saturación y el anti-windup, con el estado del integrador residiendo en la memoria del microcontrolador. Todo lo demás sigue siendo un modelo dentro de Simulink: la planta FOPDT, el tiempo muerto, la temperatura ambiente y la referencia. No intervienen el sensor ni el calentador de la TCLab.

El banco `TCLab_Test_PIL.slx` emplea de nuevo dos ramas que referencian `Controller.slx`, una en modo Normal y otra en modo PIL. A diferencia del banco SIL, las dos ramas comparten solo las fuentes T_{ref} y T_{amb}, y cada una conserva su propia copia del retardo y de la planta. Esa separación es necesaria porque, si las ramas compartieran la planta, cualquier discrepancia entre las acciones de control produciría trayectorias de temperatura distintas y dejaría de poder atribuirse la diferencia a una sola causa. Tampoco se mantuvieron simultáneamente una instancia SIL y una PIL referenciando el mismo modelo.

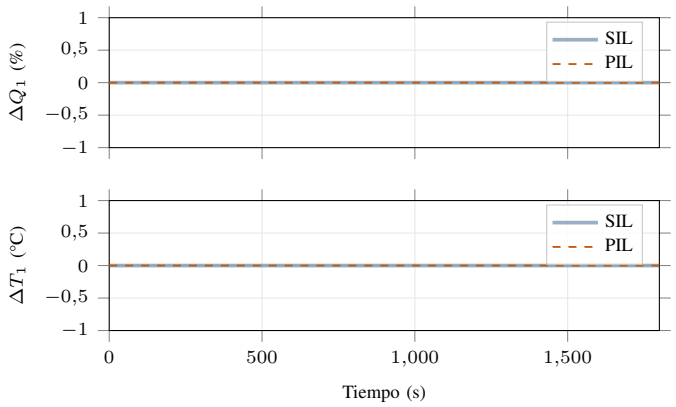


Figura 5. Diferencias *back-to-back* respecto de la ejecución Normal. Las cuatro trazas son idénticamente nulas en las 1801 muestras; la escala vertical se fijó en ± 1 para hacer visible que no hay desviación alguna.

TABLA VII
Resultados cuantitativos de las verificaciones SIL y PIL, sobre 1801 muestras ($T_s = 1$ s, 1800 s de simulación).

	SIL		PIL		Igualdad exacta	
Señal	Máx. abs.	RMSE	Máx. abs.	RMSE	SIL	PIL
Q_1 (%)	0	0	0	0	Sí	Sí
T_1 (°C)	0	0	0	0	Sí	Sí
Resultado					**PASS**	**PASS**

Las diferencias evaluadas fueron las análogas a (11), $\Delta Q_1 = Q_{1,\mathrm{PIL}} - Q_{1,\mathrm{Normal}}$ y $\Delta T_1 = T_{1,\mathrm{PIL}} - T_{1,\mathrm{Normal}}$, sobre las mismas 1801 muestras. El resultado fue también de igualdad exacta (Fig. 5 y Tabla VII). No se observó ninguna diferencia que analizar, lo cual es consistente con haber conservado el tipo `double` y la misma estructura algorítmica en las tres ejecuciones.

La evidencia que aporta PIL es distinta de la de SIL. SIL comprueba la traducción de modelo a código usando el compilador y la aritmética del computador anfitrión; PIL añade que ese mismo código se compila con la cadena de herramientas del procesador objetivo, cabe en su memoria y produce los mismos resultados ejecutándose sobre él. Es decir, PIL cubre los efectos del compilador cruzado, de los tamaños de palabra y del comportamiento aritmético del dispositivo, que SIL no puede detectar.

Aun así, un PASS en PIL no constituye una validación del controlador sobre la TCLab. Durante la prueba la planta y el retardo siguen siendo modelos, T_{amb} es una señal de Simulink, no se mide temperatura con el sensor real ni se acciona el calentador, no se ejercitan los canales físicos de entrada y salida, y la ejecución no se realiza en tiempo real contra un

proceso físico. Lo que PIL verifica es el código y su ejecución sobre el procesador, no el lazo cerrado sobre la planta.

VI. Implementación experimental

VI-A. Arquitectura del lazo físico

En esta etapa desaparece el modelo de la planta. El lazo implementado es

$$T_{\mathrm{ref}} \rightarrow \text{controlador (Arduino Leonardo)} \rightarrow Q_1 \rightarrow \text{calentador} \rightarrow T_1 \rightarrow \text{controlador},$$

y se ejecutó sobre el Arduino Leonardo integrado en la TCLab mediante Monitor & Tune [11]. El algoritmo del controlador se copió a `Controller_implementacion.slx` para no alterar `Controller.slx`, que se conservó como referencia de las etapas de verificación. La única diferencia entre ambos es la representación numérica: en la versión de implementación las entradas, las ganancias, los límites y el estado del integrador se resolvieron en `single`, adecuado para el microcontrolador de 8 bits, mientras que la ley de control, la saturación, el anti-windup y $T_s = 1\,\mathrm{s}$ se mantuvieron sin cambios.

La cadena de señal es la siguiente. La temperatura se lee del canal analógico `A0`, se convierte a `single` y se escala mediante una ganancia $500/1023$ seguida de un desplazamiento de $-50\,^{\circ}\mathrm{C}$, correspondientes al rango del sensor y a la resolución del convertidor de 10 bits. La medición pasa por un filtro de media móvil con $\lambda = 0{,}5$ antes de entrar al controlador. La salida se satura, se escala a la resolución del PWM y se convierte a `uint8` para el pin `D3`, que comanda el calentador.

Es importante precisar el papel de Monitor & Tune. Al iniciarlo, Simulink genera el código del modelo y sus modelos referenciados, lo compila y lo descarga al Arduino [12]. Todas las operaciones del lazo —adquisición de T_1, conversión, filtrado, ejecución del PI, cálculo y limitación de Q_1 y generación del PWM— se ejecutan sobre el Arduino Leonardo. La computadora no participa en el cálculo del lazo de control: el enlace de *External Mode* sirve para monitorizar señales, registrar resultados y ajustar parámetros durante la ejecución.

VI-B. Resultados experimentales

Tras una ejecución previa con $T_{\mathrm{ref}} = 25\,^{\circ}\mathrm{C}$ para comprobar la lectura del sensor y los límites del actuador, el experimento definitivo se realizó con $T_{\mathrm{ref}} = 60\,^{\circ}\mathrm{C}$ durante 607 s (608 muestras), partiendo de $T_1 = 26{,}25\,^{\circ}\mathrm{C}$. La Fig. 6 presenta las señales registradas junto con la respuesta MIL, y la Tabla VIII compara ambas cuantitativamente.

El controlador cumple su función sobre la planta real: alcanza la referencia, la mantiene y respeta en todo momento el intervalo de potencia permitido, con un máximo del 90 % durante el calentamiento inicial y sin saturación sostenida injustificada. El error estacionario, calculado como la diferencia entre la referencia y la temperatura media para $t \geq 300\,\mathrm{s}$, es de 0,04 °C. Ese valor equivale a menos de una décima parte del escalón mínimo del convertidor, $500/1023 = 0{,}489\,^{\circ}\mathrm{C}$; es decir, el error queda muy por debajo de la resolución con que el sistema puede medir la temperatura.

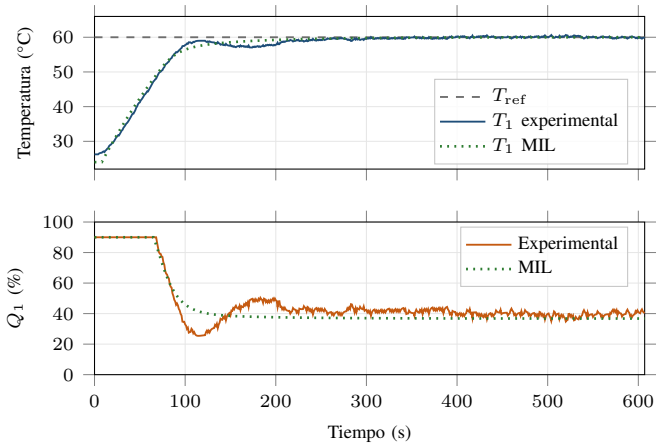


Figura 6. Implementación sobre la TCLab con $T_{\mathrm{ref}} = 60\,^{\circ}\mathrm{C}$, superpuesta a la respuesta MIL del mismo controlador.

TABLA VIII
DESEMPEÑO EXPERIMENTAL FRENTE AL OBTENIDO EN MIL. LAS ETAPAS SIL Y PIL REPRODUJERON EXACTAMENTE EL RESULTADO MIL.

Métrica	**MIL**	**Experimental**
Tiempo de subida (s)	84,5	74
t_s (s)	205,0	298
M_p (%)	0,00	1,96
e_{ss} (°C)	0,00	0,04
$T_{1,\mathrm{máx}}$ (°C)	60,0	60,66
Q_1 mínimo aplicado (%)	36,8	25,42
Q_1 en régimen (%)	36,8	40,3
Tiempo con Q_1 saturada (s)	66	69

Las diferencias respecto de MIL son sistemáticas y admiten interpretación. El sobreimpulso de 1,96 % merece una precisión: el máximo absoluto del registro, 60,66 °C, no ocurre durante el transitorio sino en $t = 520\,\mathrm{s}$, ya en régimen permanente, mientras que el máximo alcanzado antes de que la respuesta se establezca, en $t = 273\,\mathrm{s}$, es de 59,80 °C, por debajo de la referencia. Lo que la métrica está recogiendo es la oscilación de baja amplitud alrededor del punto de operación —la desviación típica de T_1 para $t \geq 300\,\mathrm{s}$ es de 0,245 °C— y no un sobredisparo de la respuesta al escalón.

La discrepancia más informativa está en el régimen permanente. En MIL el lazo se estabiliza con $Q_1 = 36{,}8\,\%$, mientras que la planta real necesita 40,3 % para sostener la misma temperatura. La ganancia estática aparente durante el experimento resulta entonces de aproximadamente 0,84 °C/%, cerca de un 15 % por debajo del $K = 0{,}979\,^{\circ}\mathrm{C}/\%$ del modelo nominal. Este error de ganancia explica también el tiempo de establecimiento mayor: con menos ganancia efectiva de planta, el lazo cierra más lento, y el sistema tarda 298 s en lugar de 205 s en entrar en la banda del 2 %. El tiempo de subida, en cambio, resultó menor que en MIL (74 s frente a 84,5 s), lo que es coherente con haber partido de 26,25 °C en lugar de 24 °C: el escalón real fue un 6 % más pequeño.

Otras diferencias visibles en la Fig. 6 no aparecen en MIL por construcción. La señal Q_1 experimental es ruidosa en régimen (desviación típica de 1,81 %) porque el controlador amplifica el ruido de medición por $K_p = 7{,}62\,\%/^\circ\mathrm{C}$, de modo que un solo escalón del convertidor produce una variación de casi cuatro puntos porcentuales de potencia. El descenso hasta el 25,4 % en $t = 115\,\mathrm{s}$, más profundo que el 36,8 % de MIL, corresponde a la corrección tras la saturación inicial sobre una planta cuya dinámica real no coincide exactamente con el FOPDT identificado.

VII. Discusión

El recorrido completo deja ver que cada etapa del flujo responde una pregunta distinta, y que ninguna sustituye a las demás.

El modelo obtenido del balance de energía resultó útil para entender el sistema pero no para diseñar. Su linealización es válida solo en un entorno reducido del punto de operación, como cuantifica la Tabla I, mientras que el transitorio que debe manejar el controlador es precisamente un calentamiento desde temperatura ambiente. Por eso el modelo de diseño tuvo que obtenerse de forma experimental. El FOPDT identificado con el ensayo escalón cumple esa función, pero la validación cruzada mostró también su límite: los residuos no son ruido alrededor de cero sino un patrón que crece con el nivel de potencia. Esto significa que el modelo no falla al azar, sino que subestima la temperatura de forma creciente a medida que se aleja de la región donde fue identificado, y que un solo modelo lineal con retardo no describe la planta en todo el rango. La elección del modelo nominal fue, en ese sentido, una elección de región de operación más que de calidad absoluta.

El efecto de la saturación resultó ser el factor de diseño más relevante de toda la práctica. El PI sintonizado con SIMC demanda un 274 % de potencia en el primer instante, casi tres veces lo que el actuador puede entregar. Con el actuador limitado pero sin protección del integrador, el controlador sintonizado correctamente incumple el requisito de sobreimpulso por un factor de seis. La causa no está en la sintonización sino en que el integrador sigue acumulando mientras la potencia ya no puede crecer, de modo que el estado del controlador deja de corresponder a lo que realmente ocurre en la planta. El *clamping* corrige eso deteniendo la acumulación durante la saturación, y el costo es de apenas 10 s de tiempo de subida. La lección es que el desempeño de un controlador no puede evaluarse sin las restricciones del actuador: el caso ideal de la Tabla V cumple los requisitos y a la vez es irrealizable.

La comparación entre los dos diseños refuerza esa idea. Sobre el modelo lineal sin restricciones el PI–CSD parece claramente superior, con mejor tiempo de establecimiento, menor sobreimpulso y márgenes mayores. Al incorporar el límite de potencia el orden cambia parcialmente: el PI–SIMC resulta más rápido porque aprovecha toda la autoridad disponible, mientras que el PI–CSD se mantiene mucho más cerca de la capacidad real del actuador. Ambos cumplen, y decidir entre ellos exige un criterio explícito —rapidez frente a robustez y esfuerzo de control— que no se deduce de las métricas por sí solas.

Las verificaciones SIL y PIL cumplieron un papel distinto del de MIL. MIL evalúa si el algoritmo resuelve el problema de control; SIL y PIL no evalúan desempeño en absoluto, sino conservación de comportamiento al cambiar de representación. Que ambas dieran igualdad exacta y no solo equivalencia dentro de una tolerancia da confianza en que las diferencias observadas después, sobre la planta real, no provienen del código ni del procesador. Ese es su valor práctico: al llegar a la implementación física, el conjunto de causas posibles de una discrepancia ya está acotado al modelo de la planta y al hardware de entrada y salida. Al mismo tiempo, conviene no sobreinterpretar el resultado: durante PIL la planta, el retardo y la temperatura ambiente siguen siendo señales de Simulink, y no se ejercita ningún canal físico.

Efectivamente, las diferencias aparecieron al cerrar el lazo sobre la TCLab, y son del tipo que las etapas anteriores no podían revelar. La más significativa es el error de ganancia: la planta necesita en régimen más potencia que la prevista (Tabla VIII), lo que equivale a una ganancia estática aparente cerca de un 15 % menor que la nominal. Esa diferencia es coherente con el sesgo detectado en la validación cruzada y con la dependencia de la ganancia respecto de la región de operación, y se traduce en un lazo más lento que el simulado. A ello se suman efectos ausentes del modelo: el ruido de medición amplificado por $K_p = 7{,}62\,\%/^\circ\mathrm{C}$, la cuantización del convertidor, y variaciones ambientales durante los 607 s del experimento. Ninguno de estos efectos invalida el diseño —el controlador cumple los requisitos sobre la planta real— pero sí muestran que el margen con que se cumplen es menor que el que sugería la simulación, y que ese margen proviene en buena medida de haber elegido un τ_c conservador.

VIII. Conclusiones

1. El modelo FOPDT identificado con el ensayo escalón, $G_{p,\mathrm{nom}}(s) = 0{,}979\,e^{-8{,}00s}/(179{,}1s + 1)$, representa adecuadamente la planta en la región de operación del controlador, con un ajuste del 88,7 % sobre datos no empleados en su identificación, frente al 74,4 % del modelo multinivel. Su limitación es igualmente clara: el residuo de validación no es aleatorio sino creciente con el nivel de potencia, de 0,88 °C a 2,20 °C entre $Q_1 = 20\,\%$ y 50 %, lo que indica que una sola ganancia no describe todo el rango de la planta.
2. El anti-windup, y no la sintonización, fue lo que determinó el cumplimiento de las especificaciones. Con el mismo K_p y el mismo T_i obtenidos por SIMC, la configuración sin protección del integrador produce $M_p = 30{,}1\,\%$ e incumple el requisito $M_p \leq 5\,\%$ por un factor de seis, mientras que con *clamping* se obtiene $M_p = 0\,\%$ y $t_s = 205\,\mathrm{s}$. Haber aplicado correctamente las reglas SIMC no demuestra por sí mismo que el controlador cumpla los requisitos.
3. Los dos diseños PI cumplen las especificaciones en MIL, pero con compromisos opuestos y sin que ninguno domine al otro: el PI–SIMC establece 47 s antes, y el PI–CSD

demanda un comando inicial de 108 % frente a 274 % y mantiene el actuador saturado 21 s frente a 66 s, además de contar con márgenes de estabilidad mayores. La selección depende del criterio que se privilegie y no se deduce de las métricas de desempeño por sí solas.

4. Las verificaciones SIL y PIL resultaron en igualdad exacta de Q_1 y T_1 sobre 1801 muestras, lo que acredita que ni la generación de código ni su ejecución sobre el Arduino Uno R4 Minima alteraron el comportamiento del controlador validado en MIL. Ese resultado no constituye una validación del sistema completo: durante ambas pruebas la planta, el retardo y la temperatura ambiente permanecen simulados y no se ejercitan los canales físicos de la TCLab, de modo que el valor de estas etapas está en acotar las causas posibles de una discrepancia posterior, no en anticiparla.
5. La implementación sobre la TCLab cumple los requisitos —$t_s = 298\,\mathrm{s}$ y $M_p = 1{,}96\,\%$— con un error estacionario de 0,04 °C, muy inferior al escalón mínimo del convertidor (0,489 °C). Las diferencias respecto de MIL se explican principalmente por un error de ganancia del modelo: la planta requiere 40,3 % de potencia en régimen frente al 36,8 % previsto, lo que corresponde a una ganancia estática aparente cerca de un 15 % menor que la nominal y explica el tiempo de establecimiento un 45 % mayor.

Referencias

[1] MathWorks, "What is hardware-in-the-loop (HIL)?" disponible en: https://www.mathworks.com/discovery/hardware-in-the-loop-hil.html. Consultado: 1 de septiembre de 2026.

[2] J. D. Hedengren, "Arduino dynamic temperature modeling," APMonitor: Dynamics and Control, disponible en: https://apmonitor.com/pdc/index.php/Main/ArduinoModeling. Consultado: 11 de agosto de 2026.

[3] ——, "TCLab linearize energy balance," APMonitor: Dynamics and Control, disponible en: https://apmonitor.com/pdc/index.php/Main/TCLabLinearize. Consultado: 11 de agosto de 2026.

[4] ——, "TCLab FOPDT model," APMonitor: Dynamics and Control, disponible en: https://apmonitor.com/pdc/index.php/Main/TCLabFOPDT. Consultado: 11 de agosto de 2026.

[5] S. Skogestad, "Simple analytic rules for model reduction and PID controller tuning," *Journal of Process Control*, vol. 13, no. 4, pp. 291–309, 2003.

[6] MathWorks, "Anti-windup control using a PID controller," Simulink Documentation, disponible en: https://www.mathworks.com/help/simulink/slref/anti-windup-control-using-a-pid-controller.html. Consultado: 12 de agosto de 2026.

[7] ——, "stepinfo: Rise time, settling time, and other step-response characteristics," Control System Toolbox Documentation, disponible en: https://www.mathworks.com/help/control/ref/dynamicsystem.stepinfo.html. Consultado: 12 de agosto de 2026.

[8] ——, "Source code generation," Embedded Coder Documentation, disponible en: https://www.mathworks.com/help/ecoder/source-code-generation-and-verification.html. Consultado: 1 de septiembre de 2026.

[9] ——, "Software-in-the-loop simulation," Embedded Coder Documentation, disponible en: https://www.mathworks.com/help/ecoder/software-in-the-loop-sil-simulation.html. Consultado: 1 de septiembre de 2026.

[10] ——, "Processor-in-the-loop simulation," Embedded Coder Documentation, disponible en: https://www.mathworks.com/help/ecoder/processor-in-the-loop.html. Consultado: 1 de septiembre de 2026.

[11] ——, "Arduino hardware," Simulink Support Package for Arduino Hardware Documentation, disponible en: https://www.mathworks.com/help/simulink/supportpkg-arduino.html. Consultado: 1 de septiembre de 2026.

[12] ——, "Prepare, configure, and run simulink model for different user workflows on arduino," Simulink Support Package for Arduino Hardware Documentation, disponible en: https://www.mathworks.com/help/simulink/supportpkg/arduino_ref/prepare-configure-and-run-simulink-model-for-different-user-workflows-on-arduino.html. Consultado: 1 de septiembre de 2026.